



TechCouncil
of Australia

Agentic Engineering

Student Handout

Student Handout — TCA Agentic Engineering

Everything here works in **Claude Code** and **OpenAI Codex**. Type the prompts in verbatim; where a mechanism differs, both are shown. You are building this app yourself, from an empty folder. The reference repo is your safety net, not your starting point.

Before the day

Ten minutes, once. The day has no time to debug a missing toolchain, and the safety net does not help if the agent will not start.

TERMINAL

```
# One agent, or both. Either is fine; the ideas are identical.
npm install -g @anthropic-ai/claude-code && claude # then log in
npm install -g @openai/codex && codex # then sign in

# Everything below must print a version, not an error.
claude --version # or: codex --version
node --version # 18+, and Codex prefers 22+
python3 --version # 3.11+
git --version
uv --version # https://docs.astral.sh/uv/
```

- Fifteen minutes of reading: *the first two lessons of Claude Code 101, or the Codex CLI quickstart*. You want the vocabulary, not mastery: agentic loop, context window, permissions, approvals.
- **One sentence to bring:** a habit you actually want to build. You will seed the app with it, which makes the day yours rather than a toy.
- Environment stale after time away? Old Node, expired credentials, a wiped laptop: flag it the day before, not at 09:35.

Two folders. Know which is which

	Where	What it is
Your app	~/my-habits	Empty at 10:00. You build the whole thing here, by prompt. This is the one that matters.
The net	~/tca-habits-reference	A clone of the reference repo. You only open this if you fall behind.

Clone and warm up the net before the day so it is ready when you need it. It takes two minutes then, and five stressful minutes at 11:20.

TERMINAL

```
git clone https://github.com/paulkeen/tca-habits.git tca-habits-reference
cd tca-habits-reference
scripts/setup.sh          # backend venv (uv) + frontend deps (npm)
git tag                   # checkpoint-0-empty checkpoint-1-working
                           # checkpoint-2-features checkpoint-3-hardened
```

The safety net — reset to a checkpoint any time

Each checkpoint is the reference app at the end of a section. Reaching for one is the design, not a failure.

TERMINAL

```
cd ~/tca-habits-reference
git stash                # park whatever mess exists
git checkout checkpoint-1-working # or -2-features / -3-hardened
./scripts/setup.sh       # first time on a new checkpoint
./scripts/dev.sh         # backend :8000, frontend :5173
```

Use the scripts, not make: the Makefile only exists from checkpoint-2-features onwards. From there, make dev and make check work too. In *your* app they work from the first prompt, because you ask for them.

The reference app is fuller than what you will build in a 55-minute lab: it has had more days spent on it. Treat it as a working example to get unstuck in, not as a score to match.

Launch the agent

TERMINAL

```
cd ~/my-habits && claude    # Claude Code
cd ~/my-habits && codex     # OpenAI Codex
```

The cockpit — four moves on the context window

The move	Claude Code	Codex
SEE what it is holding right now	/context	/status
LOAD the exact files the task touches	@path/to/file	@path/to/file
TRIM summarise away what is stale	/compact	/compact
RESET new task, clean window	/clear	/new

See, load, trim, reset. Nothing to memorise, but every context problem you hit today is one of those four. Check the gauge before a big task, and the moment answers go vague.

Command map

Want to...	Claude Code	Codex
Clean slate	<code>/clear</code>	<code>/new</code>
Summarise a long session	<code>/compact</code>	<code>/compact</code>
See what fills the context	<code>/context</code>	<code>/status</code>
Token usage / cost	<code>/usage</code>	<code>/status</code>
Switch model or reasoning effort	<code>/model</code>	<code>/model</code>
Plan before touching files	Shift+Tab to plan mode, or <code>/plan</code>	<code>/plan</code>
Scaffold the project contract	<code>/init</code> (writes CLAUDE.md)	<code>/init</code> (writes AGENTS.md)
Review your local diff	<code>/code-review</code>	<code>/review</code>
Review a GitHub pull request	<code>/review <number></code>	<code>/review</code>
See the raw diff	ask: "show me git diff"	<code>/diff</code>
Change approvals / sandbox	Shift+Tab cycles the mode	<code>/permissions</code>
Security pass over a change	<code>/security-review</code>	<code>/review</code> , then ask for security explicitly
MCP servers	<code>/mcp</code>	<code>/mcp</code>
Sub-agents	edit <code>.claude/agents/*.md</code>	edit <code>.codex/agents/*.toml</code>
Reusable command	<code>.claude/commands/</code> or <code>.claude/skills/</code>	<code>~/.codex/prompts/</code>
Hooks	<code>/hooks</code> , or <code>.claude/settings.json</code>	<code>.codex/hooks.json</code> or <code>config.toml</code>
Undo in-session	<code>/rewind</code> , or Esc Esc on an empty prompt	git + resume
Headless, for scripts and CI	<code>claude -p "..."</code>	<code>codex exec "..."</code>
Pin a file into context	<code>@path/to/file</code>	<code>@path/to/file</code>
Help	<code>/help</code>	<code>/help</code>

Three traps. Shift+Tab *cycles* through modes rather than jumping to plan, so it is often two presses. Esc Esc only rewinds when the prompt box is empty; with text in it, it clears the box instead. And /review in Claude Code now means a GitHub pull request, so reach for /code-review when you mean the diff on your machine.

Both tools ship changes most weeks. This map was checked against the docs in July 2026, and the fastest way to check it yourself is to ask the agent: *"what is the current command for X in the version I am running?"*

Two agents at once (git worktrees)

A worktree gives a branch its own directory, so two agents work the same repository without fighting over files. Name the branch and the folder the same thing or you will lose track of which terminal you are in.

TERMINAL

```
git worktree add ../my-habits-nudge -b nudge    # its own folder AND branch

cd ~/my-habits          && claude    # Terminal 1, branch main
cd ../my-habits-nudge   && codex    # Terminal 2, branch nudge
#   give both the same prompt, then compare:
git diff main nudge --stat

git merge nudge          # or cherry-pick the good bits
git worktree remove ../my-habits-nudge && git branch -d nudge
```

The -b matters. Without it git expects the branch to already exist and refuses with invalid reference.

The prompts

Copy these as they are, or better, adapt them. Every one is written the way the day teaches: an outcome, the constraints that matter, and no filenames or implementation steps. The tell that you have got it right is that the same words work in either tool.

Section 1 · Vibe Coding

Block A — stand the whole thing up (15 min)

TERMINAL

```
mkdir my-habits && cd my-habits && git init
claude                # or: codex
```

PROMPT

Build a habit tracker as a full-stack app in this empty folder.

Backend: FastAPI with SQLite via SQLAlchemy. Two tables: habits (id, name, emoji, created_at) and completions (id, habit_id, completed_date stored as YYYY-MM-DD). Endpoints: GET /habits returns every habit with today's completion status and its current streak; POST /habits creates one; POST /habits/{id}/complete marks today done and is idempotent; DELETE /habits/{id}/complete unmarks today. A streak is the number of consecutive days with a completion counting back from today, and today not being ticked yet must not break it.

Frontend: React, TypeScript, Vite. One page listing each habit with its emoji, a tick control and a streak badge, plus a form to add a habit. Proxy the API through the Vite dev server so I never think about CORS.

Seed it with these habits: <add your three habits>

Also add scripts/setup.sh and scripts/dev.sh that install and run both servers, and a Makefile whose setup, dev and test targets call them.

Show me a plan before you write anything.

Read the plan like a junior's design doc. Is the streak logic where you would put it? Did it understand the real goal, or a plausible lookalike? What did it assume that you never said? Correct it there, where changes are free, then approve and watch the tools fire.

TERMINAL

```
make dev                # or whatever run command it printed
# backend :8000, frontend :5173. Ctrl-C stops both.
```

Block B — make it run, and react (15 min)

It will not be perfect. Practise the recovery loop: report symptoms, do not open the file.

PROMPT

The app won't start (or the page is blank). Here is the terminal output and what I see on screen:

<paste the full error, then one line describing the screen>

Diagnose and fix it. Don't ask me to open files, you have them.

Then exercise it by hand: tick a habit and watch the streak, untick it and watch it reset. If the maths looks wrong, describe what you saw rather than what to change.

PROMPT

I ticked <habit> and the streak still shows 3. Yesterday and today are both ticked in the UI. Tell me what you think is happening before you change anything.

Block C — one real feature, your way (15 min)

Phrase it purely as an outcome. Notice you never name a file or a function.

PROMPT

I want to see the date each habit was last completed, shown next to its name.

PROMPT

Add a longest streak alongside the current one, and make the streak badge stand out once it passes seven days.

Block D — commit (5 min)**PROMPT**

Commit everything with git and a message describing what changed.

You just shipped without writing a line. If the agent lacks git permission, the equivalent by hand is `git add -A && git commit -m "feat: working habit tracker"`.

Stretch, if you are flying

- Ask the agent to explain its own streak calculation back to you, then find the timezone bug by conversation alone.
- Make the badge pulse with CSS above a thirty-day streak.
- Add a delete button, and ask what should happen to the completions.

Section 2 · Vibe Engineering**Block A — write the contract (10 min)**

PROMPT

Create AGENTS.md as the single project contract for this app. Include:

- the commands to set up, run and test it, exactly as they work here;
- the architecture: which file holds what, in five lines;
- one architectural decision and why: streaks are computed fresh from the completion rows on every request, never stored;
- one hard rule you must never break: never change the streak calculation without adding or updating a test for it.

Then make CLAUDE.md a one-line import of it: @AGENTS.md

Keep it under sixty lines. A contract nobody reads is decoration.

Shortcut: /init scaffolds a contract in both tools. Keep AGENTS.md as the source of truth: Codex reads it directly, Claude Code reads it through the import in CLAUDE.md. Then prove it works, in a *fresh* session:

PROMPT · PROOF THE CONTRACT IS READ

Without me repeating myself: what is this project's rule about the streak calculation, and where does the streak get computed?

Block B — codify the workflow (10 min)**PROMPT**

Turn my add-a-feature recipe into a reusable command called add-feature, wired for the tool I'm running. The body:

1. Explore the files this feature touches; report what you found in three lines.
2. Plan it briefly, and surface any real decision instead of deciding silently.
3. Implement it following AGENTS.md.
4. Run the app or the tests and prove it works.
5. Summarise with git diff --stat: what you verified, what you left out.

The feature description arrives as an argument.

Claude Code: .claude/commands/add-feature.md, or as a skill in .claude/skills/add-feature/SKILL.md, which is now the recommended home and gives the same /add-feature.

Codex: ~/.codex/prompts/add-feature.md. Tell me if that mechanism has been superseded in the version I'm running.

Note: Claude session will need to be restarted.

Then use it on a refactor, where the Explore step earns its keep:

PROMPT

/add-feature split backend/main.py into models.py for the SQLAlchemy models, schemas.py for the Pydantic schemas, and database.py for the engine, the session, and a run_migrations() that adds any missing columns at startup. Behaviour must not change: same endpoints, same responses. Prove it by running the app and ticking a habit.

Block C — grow the app with your own command (15 min)

Two invocations, one prompt each. The first is the one the afternoon depends on, so do it even if you skip the second.

PROMPT

```
/add-feature a habit should only be due on the days I choose. Add target days per habit, stored as a list of integers where 0 is Sunday and 6 is Saturday, defaulting to every day. A streak must count only the days a habit was actually due: missing a day it was never scheduled for must not break the chain. Add the migration for the new column, and show me in the UI which habits are due today. Before you code, tell me which weekday convention Python's date.weekday() uses and how you will convert between the two.
```

PROMPT

```
/add-feature give each habit a colour from a fixed palette of nine, chosen when I create it, and show habits as a grid of tappable tiles that fill with their colour when complete, instead of a plain list. Keep every existing behaviour identical, and add the migration.
```

If you have time left, this one is pure joy for four minutes of work:

PROMPT

```
/add-feature a light and dark theme with a toggle in the header, remembered between visits, and no flash of the wrong theme when the page loads.
```

Block D — a feature that needs judgement (10 min)

Turn plan mode on first (Claude Code: Shift+Tab to plan mode. Codex: /plan), so the agent has to surface the decision instead of making it while you look away.

PROMPT

```
Plan a weekly summary feature: a short, human paragraph summarising the week. Before writing any code, lay out where the text should come from, a deterministic template versus a model (LLM) call, with the trade-offs of each, and recommend one. It must still work with no API key set. Wait for my decision before implementing.
```

PROMPT · AFTER YOU DECIDE

```
Implement the option I chose. If it calls a model, that path stays optional: no API key means the template, and a failed call must never block the page. Return a field on the response saying which one produced the text, so the UI can badge it.
```

Block E — parallel, and a second opinion (15 min)

Set up the worktree with the commands on the previous page, then give **both** agents exactly this:

PROMPT

Add an AI encouragement feature: when a habit is due today, is not yet done, and its current streak is greater than zero, surface a short supportive nudge to keep the streak alive. It must fall back to a fixed template when no API key is set, and never block the page if the model call fails.

TERMINAL

```
git diff main nudge --stat      # two takes on the same task
```

Which would you merge, and why? That question is the whole exercise. Commit each feature separately so the parallel work stays mergeable.

Stretch

- A stats view: total completions, how many habits have a live streak, the longest streak ever, and this week's completion rate as a bar per day.
- Connect a real MCP server, your calendar for instance, and have the agent suggest habit times around your actual schedule.
- Have Claude Code and Codex review each other's encouragement feature, then reconcile the two reviews yourself.

Section 3 · Agentic Engineering

Block A — build the harness (15 min)

First make one command that says yes or no about the whole backend. Without it, nothing else this afternoon has anything to stand on.

PROMPT

Give me one command that says yes or no about this whole backend. Add ruff for lint and mypy for types to the dev dependencies, then a `make check` target that runs lint, then types, then the tests, stopping at the first failure. Fix whatever the first run finds. Don't relax the rules to make it pass.

PROMPT

Write a pytest suite for the streak logic, covering the edge cases we skipped this morning:

1. a missed scheduled day resets the streak to zero;
2. completing the same habit twice in one day is idempotent, one completion;
3. a day still in progress doesn't break a streak: today not ticked yet must not zero it;
4. the weekday conversion between my 0=Sunday convention and Python's Monday=0.

Test the pure functions directly so the suite is deterministic whatever day it runs on. Run them, show me one failing, then make it pass.

Now the honest one. This is the most useful ninety seconds of the afternoon:

PROMPT · THE TRAP

Be honest with me: does anything in this app know what timezone I'm in? Show me every place where 'today' gets decided. Then write one more test, marked xfail with a reason, that would fail if the app got midnight wrong for a user in another timezone. Don't fix it. I want the gap recorded, not hidden.

Block B — define the sub-agents (15 min)**PROMPT**

Create a read-only "reviewer" sub-agent for this repo. Its only job: read a git diff and report problems as a numbered list with file:line and a severity of blocker, should-fix or nit. It looks for correctness bugs in the date and streak maths, weekday-convention mistakes, idempotency holes, missing tests, and anything that breaks AGENTS.md. It must never edit a file: give it the minimum tools to read and search only. Wire it for the tool I'm running and tell me where the file lives.

PROMPT · THE OTHER TWO

Now the same for two more. A "test-writer" that writes failing tests from a requirement and never implements. A "verifier" that runs `make check` and the app, then reports green or exactly why not. Narrow prompts, minimum tools.

Note: Claude session will need to be restarted.

Block C — run the agent team (20 min)**PROMPT**

Feature: a monthly progress ring. Per habit, the percentage of its scheduled days this month that were completed, shown in that habit's detail view.

Drive it as a team: (1) the test-writer writes failing tests first; (2) implement until they pass; (3) the reviewer critiques the diff and you address what it raises; (4) the verifier runs `make check` and the app and confirms green. Don't ask me to type code. I'll arbitrate.

If the team stalls, the fix is almost never a better prompt. It is a clearer definition of done.

Block D — lock it in (10 min)**PROMPT**

Add two guardrail hooks, wired for the tool I'm running: a Stop hook that runs `make check` at the end of every turn and refuses to finish while it's red, and a PreToolUse hook that blocks any edit to the streak logic unless a test file changes in the same turn. The blocking hook must exit 2, not 1, and its stderr must tell me what to do about it.

PROMPT · PROVE IT

Now prove the guardrail works. Try to change the streak calculation without touching a test, and show me the block message. Then tell me what would have happened if that hook exited 1 instead of 2.

PROMPT

Add a GitHub Actions workflow at `.github/workflows/ci.yml` that runs ``make check`` and the frontend build on every push and pull request.

TERMINAL

```
# Claude Code
/security-review      # the security pass
/code-review         # the local diff, correctness and style

# Codex
/review      then: "review this diff for security, secret-leak and licensing issues"

git add -A && git commit -m "chore: harden - tests, hooks, CI"
git tag my-hardened-app
```

Stretch

- Build a five-case eval for the weekly summary and have a sub-agent grade the outputs against the rubric.
- Add a docs-writer sub-agent that updates the README from the diff.
- Run the whole team against a Codex-built branch and a Claude-built branch, then reconcile them into one merge.
- Make the app genuinely timezone-correct, and turn that xfail test green.

Golden rules

Section 1 · flying the loop

- **Resist the keyboard.** Direct the loop; don't type the code.
- **Prompt by outcome.** Say what, and the constraints. Never how, and never which file.
- **Plan review, not plan writing.** Read for intent and scope, not mechanics. A tidy plan aimed at the wrong problem is the dangerous one.
- **Recover by reporting symptoms.** Paste the error, describe the screen. Don't hand-debug.
- **Commit small; use the net.** git is your undo. The checkpoint is the room's.

Section 2 · making it repeatable

- **Write it down once.** The contract stops you repeating yourself.
- **Save your best prompts.** A good prompt becomes a command, not a memory.
- **Keep a clean desk.** One task, one session. Trim before it bloats.
- **Get a second opinion.** Two agents beat one. Compare before you merge.
- **Make the agent surface decisions.** Plan first. You arbitrate; it doesn't decide quietly.

Section 3 · engineering the trust

- **Harness first.** Write the check before the feature. Green is the definition of done.
- **Fail it once.** A test that has never failed is decoration, not proof.
- **Narrow the agents.** One job, minimum tools. The reviewer never gets write access.
- **Orchestrate, don't type.** Pick the work, settle the disputes, decide when it's finished.
- **Enforce it in the tool.** Hooks and CI, not good intentions. A contract is a promise; a hook is a fact.

Spec, not step list

When a task is big enough to need writing down, five lines beat five paragraphs. Experienced engineers are already good at this, and the skill transfers straight in.

THE FIVE LINES

Objective:	what must be true when this is done
Constraints:	what it must not break, and what it must work without
Key decisions:	the choices you want surfaced, not made quietly
Verification:	how we will know, in a command or a test
Out of scope:	what you are deliberately not doing today

On Monday, in your own repo

Today was a toy app with facilitators in the room. This is the order to do it in on work that matters.

When	Do exactly this	The trap
Tomorrow	One repo you own. One command: lint, types, tests. Watch it fail once.	Designing the perfect harness for six weeks and shipping nothing. Nothing clever.
This week	One read-only reviewer sub-agent, plus one hook that blocks the mistake your team keeps making.	Trying to wire all six hooks at once, and bouncing off.
This month	Harness into CI, an eval for anything fuzzy, and walk one teammate through the same two steps.	Keeping it as a personal habit. It has to survive you going on leave.

Which repo? Not the greenfield side project, and not the terrifying monolith. The one where reviewing the diff already makes you uneasy. That discomfort is a measurement, and it is telling you where the harness has the most work to do.

Worth bookmarking

- code.claude.com/docs · commands, sub-agents, hooks, skills, permission modes
- developers.openai.com/codex · install, auth, config.toml, sandbox modes, subagents, hooks
- [agents.md](#) · the cross-tool project contract both agents read
- Anthropic's free courses: Claude Code 101, and Claude Code in Action.

These pages move. Trust them over any deck, including this one.